GitOps with Argo CD

Декларативные деплои через Git как single source of truth

DevOps Pro — Kubernetes Advanced

Companion lab: `lab-gitops-argocd`

Agenda

Part 1: Зачем GitOps

Проблема релизов и принципы GitOps

Part 2: Архитектура Argo CD

Компоненты и поток данных

Part 3: Application и sync

Как Argo приводит кластер к Git

Part 4: Drift, self-heal, rollback

Самовосстановление и откаты

Part 5: Структура репозиториев

App-of-apps, ApplicationSets, окружения

Part 6: Безопасность, практики, Argo vs Flux

Part 1: Зачем GitOps

От «деплой руками» к «Git — источник истины»

Проблема: как обычно катят в кластер

Императивно

- `kubectl apply` с ноутбука
- `helm install` из CI runner-а
- кто-то поправил «на горячую»

Чем больно

- нет единого состояния истины
- дрейф: кластер ≠ репозиторий
- «а кто это поменял?» — не видно
- откат = вспоминать руками

> Чем больше кластеров и команд, тем сильнее расходятся «что в Git» и «что в кластере». Аудит и воспроизводимость теряются.

GitOps в одном предложении

> **Желаемое состояние системы описано декларативно в Git, а агент в кластере непрерывно приводит реальность к этому описанию.**

- **Git** — единственный источник истины (не кластер, не CI)
- Любое изменение = **commit / PR** (review, история, аудит «из коробки»)
- **Агент** в кластере сам синхронизирует и чинит дрейф

4 принципа GitOps (OpenGitOps)

📜 Declarative

вся система описана декларативно — желаемое состояние, не скрипты

🏷️ Versioned & immutable

состояние в Git: версии, история, откаты

⬇️ Pulled automatically

агент сам забирает изменения из Git

🔄 Continuously reconciled

постоянно сверяет actual vs desired и чинит дрейф

Push vs Pull

❌ Push (классический CI/CD)



креды кластера в CI · дрейф не виден

✅ Pull (GitOps)



креды не выходят наружу · дрейф чинится

> GitOps = **pull-модель**: агент в кластере, а не CI снаружи, применяет изменения. Это и безопаснее, и устраняет дрейф.

Что даёт GitOps

1 commit
= 1 изменение, с review и аудитом
git revert
= откат за секунды
0 дрейфа
кластер всегда = Git

- Воспроизводимость: новый кластер = `apply` того же репозитория
- Onboarding: весь desired state читается в одном месте
- Disaster recovery: Git + Argo = пересборка окружения

Part 2: Архитектура Argo CD

Компоненты и поток данных

Argo CD — что это

- **Декларативный GitOps CD для Kubernetes** (проект CNCF, graduated)
- Живёт **в кластере**, тянет манифесты из Git и применяет их
- Понимает **plain YAML, Helm, Kustomize, Jsonnet** — рендерит сам
- UI + CLI (`argocd`) + полностью декларативный API (CRD)

> Argo CD сам по себе — это набор контроллеров + сервер. Конфигурируется Kubernetes-объектами (`Application` , `AppProject`).

Компоненты Argo CD

Компонент	Роль
API / Server	UI, CLI, API; RBAC, SSO
Repo Server	клонирует Git, рендерит Helm/Kustomize → манифесты
Application Controller	сравнивает desired (Git) vs live (кластер), синхронизирует
Redis	кэш состояния/рендеров
ApplicationSet Controller	генерирует Application-ы из шаблонов

Поток данных



Part 3: Application и sync

Как Argo приводит кластер к Git

`Application` — главный объект

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: {name: guestbook, namespace: argocd}
spec:
  project: default
  source:                                  # ОТКУДА брать
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD                    # ветка / тег / commit
    path: guestbook                         # папка в репозитории
  destination:                             # КУДА катить
    server: https://kubernetes.default.svc
    namespace: guestbook
  syncPolicy:
    automated: {prune: true, selfHeal: true}
```

Состояния: Sync и Health

Sync — совпадает ли с Git?

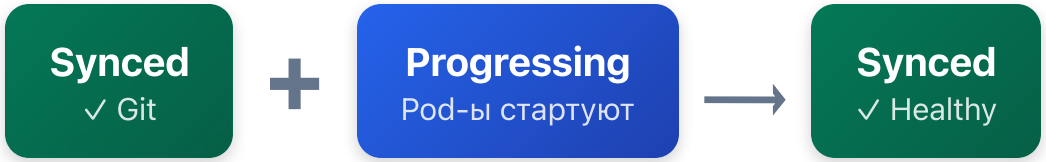
Synced кластер = Git

OutOfSync есть расхождение (+ diff)

Health — работает ли в кластере?

Healthy **Progressing**

Degraded **Missing**



> Два **независимых** измерения: **Sync** = «совпадает ли с Git», **Health** = «работает ли в кластере». `Synced + Progressing` = применили, но Pod-ы ещё поднимаются (тянут образ).

Sync: manual vs automated

Режим	Поведение
Manual	Argo показывает OutOfSync, но ждёт кнопки/ `argocd app sync`
Automated	Argo сам применяет при изменении Git
`automated.prune: true`	удалять объекты, исчезнувшие из Git
`automated.selfHeal: true`	откатывать ручные изменения в кластере

```
argocd app sync guestbook        # ручной sync
argocd app get guestbook         # статус, diff, дерево ресурсов
```

Sync waves и hooks

- **Sync waves** — порядок применения: аннотация `argocd.argoproj.io/sync-wave: "-1"` (меньше = раньше)
- **Resource hooks** — `PreSync` / `Sync` / `PostSync` (например, миграция БД до выката приложения)

```yaml
metadata:
  annotations:
    argocd.argoproj.io/hook: PreSync          # выполнить ДО основного sync
    argocd.argoproj.io/sync-wave: "-5"
```

> Waves и hooks решают зависимости: сначала CRD/namespace/секреты и миграции, потом приложения.

Part 4: Drift, self-heal, rollback

Самовосстановление и откаты

Drift detection

- Argo постоянно сравнивает **live** (кластер) и **desired** (Git)
- Любое ручное изменение (`kubectl edit/scale`) → объект помечается **OutOfSync**
- В UI/CLI виден точный **diff** на уровне полей

```
kubectl -n guestbook scale deploy guestbook-ui --replicas=5   # дрейф вручную
argocd app get guestbook        # → OutOfSync, diff показывает replicas 5 vs 1
```

> Дрейф — это норма жизни кластера. GitOps делает его **видимым** немедленно.

Self-heal



- При `selfHeal: true` Argo **сам откатывает** дрейф к состоянию из Git
- Удалили / изменили managed-объект руками → Argo **возвращает** его
- Источник истины — Git; кластер всегда подтягивается обратно

```
kubectl -n guestbook delete deploy guestbook-ui    # «случайно» удалили
# через секунды Argo воссоздаёт Deployment из Git → снова Synced/Healthy
```

> Менять надо **Git**, а не кластер. Ручное изменение живёт ровно до следующего reconcile.

Rollback = git revert

Откат «как раньше»

- вспоминать предыдущий образ
- ручной `kubectl` / `helm rollback`
- состояние не задокументировано

Откат в GitOps

- `git revert <commit>` + push
- Argo синхронизирует обратно
- история = аудит изменений

```
git revert <bad-commit> && git push      # «канонический» откат
argocd app rollback guestbook <ID>       # быстрый откат через историю Argo
```

Part 5: Структура репозиториев

App-of-apps, ApplicationSets, окружения

Config repo vs source repo

> Разделяйте **код приложения** и **манифесты деплоя** в разных репозиториях.

Source repo

- код приложения
- Dockerfile, тесты
- CI собирает образ → тег

Config repo

- манифесты / Helm / Kustomize
- то, что смотрит Argo CD
- CI бампит тег образа сюда

Окружения через overlays

```
config-repo/
├── base/                    # общие манифесты (Kustomize base)
└── overlays/
    ├── dev/    → Application: targetRevision: main,    path: overlays/dev
    ├── stage/  → Application: targetRevision: release,  path: overlays/stage
    └── prod/   → Application: targetRevision: v1.4.0,   path: overlays/prod
```

- Один base, окружения — overlays (см. lab-29 Kustomize)
- Разные окружения = разные `Application` (часто разные ветки/теги)

App-of-apps



Один **корневой** `Application` → папка с дочерними Application-ами → их ресурсы.

ApplicationSet

- Контроллер, **генерирующий** Application-ы из шаблона + генератора
- Генераторы: list, **git** (по папкам/файлам), **cluster** (по кластерам), matrix
- Один шаблон → приложение на каждый сервис / окружение / кластер

```yaml
kind: ApplicationSet
spec:
  generators:
  - git: {repoURL: ..., directories: [{path: apps/*}]}   # app на каждую папку
  template:
    # ... шаблон Application с {{path.basename}}
```

> Масштаб: «одно приложение на 30 кластеров» или «на каждую папку — свой Application» без копипаста.

Part 6: Безопасность, практики, Argo vs Flux

Безопасность GitOps

- **Pull-модель**: креды кластера не утекают в CI (агент внутри)
- **AppProject** — границы: какие repo / destination / kinds разрешены команде
- **RBAC + SSO** в Argo (кто что может синкать)
- **Секреты НЕ в Git в открытую** → Sealed Secrets / External Secrets / SOPS

> Git — источник истины, но **секреты в нём шифруются**. Plain-секрет в репо = инцидент.

Best practices

- Разделяйте **app-repo** и **config-repo**
- **prod** прибивайте к тегу/коммиту (`targetRevision`), не к `HEAD`
- Включайте `selfHeal` и `prune` **осознанно** (сначала привыкните к diff)
- Webhook вместо поллинга — мгновенный sync
- Защищайте config-repo: PR-review, branch protection, CODEOWNERS

> Дисциплина репозитория важнее самого Argo: GitOps хорош ровно настолько, насколько защищён и структурирован ваш Git.

Argo CD vs Flux

	Argo CD	Flux
Интерфейс	богатый **UI** + CLI	CLI-first (UI отдельно)
Модель	`Application` CRD, app-centric	набор контроллеров (Source/Kustomize/Helm)
Multi-tenancy	AppProject, SSO, RBAC	через native RBAC + GitRepository
Сила	визуализация, app-of-apps, UX	компонуемость, тесная интеграция с k8s
Оба	CNCF graduated, pull-модель, drift+reconcile	то же

> Выбор — про UX и модель, не про «лучше/хуже». Argo — app-centric с UI; Flux — компонуемый toolkit. Принципы GitOps у обоих одинаковы.

Hands-on: lab-gitops-argocd

```
# 1. поставить Argo CD
kubectl create namespace argocd
kubectl apply -n argocd --server-side \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# 2. завести Application на публичный repo (guestbook)
kubectl apply -f application.yaml      # source+destination+syncPolicy

# 3. наблюдать sync → Synced/Healthy
# 4. сломать руками (delete deploy) → self-heal вернёт
```

> В `lab-gitops-argocd` всё это проверяется на живом кластере, включая drift/self-heal.

Итог

GitOps = Git как источник истины + агент-реконсилятор

Декларативно, версионируемо, pull, непрерывный reconcile

Argo CD: Application → sync → drift/self-heal → rollback (git revert)

Структура: config-repo, overlays, app-of-apps, ApplicationSets

Безопасность: pull-модель, AppProject/RBAC, шифрованные секреты

Next: `lab-gitops-argocd` — руками на живом кластере